

RESEARCH

Open Access



Advanced system log analyzer for anomaly detection and cyber forensic investigations using LSTM and transformer networks

Leeladhar Chourasiya¹, Sushma Khatri¹, Umesh Kumar Lilhore², Sarita Simaiya³, Roobaea Alroobaea⁴, Abdullah M. Baqasah⁵, Majed Alsafyani⁶ and Monish Khan^{7*}

Abstract

This framework presents an innovative methodology that combines LSTM, Transformer, and GNN models to effectively capture both temporal and spatial patterns within log data, thus improving cybersecurity anomaly detection and forensic analysis. By utilizing LSTM networks, the system is able to model sequential log patterns over time, which aids in identifying hidden attack behaviors. Transformer architectures are employed to examine contextual relationships within logs, allowing for accurate, context-sensitive classification. Moreover, Graph Neural Networks (GNNs) depict logs as interconnected graphs, which facilitates the identification of coordinated multi-stage attacks from various sources. The integration of these models enables a thorough analysis of log data, simultaneously capturing dynamic temporal sequences and intricate relationships. The system autonomously correlates logs from system, network, and application sources to reconstruct attack timelines and identify emerging threats in real time. Empirical assessments on datasets such as HDFS, CICIDS, and UNSW-NB15 indicate that this integrated approach outperforms traditional methods, achieving detection accuracies of up to 98.2%, minimizing false positives, and expediting forensic investigations—thereby significantly enhancing the capabilities of automated cybersecurity monitoring and response.

Keywords System logs, Anomaly detection, Deep learning, LSTM, Transformer, Cybersecurity

*Correspondence:

Monish Khan
drkumacse@gmail.com

¹Department Computer Science and Engineering, Acropolis Institute of Technology and Research, Indore, India

²School of Computing Science and Engineering, Department of Computer Science and Engineering, Galgotias University, Greater Noida, UP, India

³School of Computer Applications & Technology, Galgotias University, Greater Noida, UP, India

⁴Department of Computer Science, College of Computers and Information Technology, Taif University, P. O. Box 11099, 21944, Taif, Saudi Arabia

⁵Department of Information Technology, College of Computers and Information Technology, Taif University, P. O. Box 11099, 21974, Taif, Saudi Arabia

⁶Department of Computer Science, College of Computers and Information Technology, Taif University, P. O. Box 11099, 21944, Taif, Saudi Arabia

⁷Arba Minch University, Arba Minch, Ethiopia

Introduction

The rapid expansion of digital infrastructures and the increasing sophistication of cyber threats have made cybersecurity a critical concern for organizations worldwide. Cyber incidents such as ransomware attacks, zero-day exploits, and advanced persistent threats (APTs) not only disrupt operations but also cause severe financial and reputational damage. Within this landscape, system logs which capture records of user activity, system events, and network communications play a pivotal role in threat detection, anomaly identification, and forensic investigation. Effective log analysis provides security analysts with actionable insights to monitor system health, detect intrusions, and reconstruct attack scenarios, thereby serving as the backbone of modern cybersecurity defence mechanisms [1]. However, traditional log analysis approaches face significant limitations in addressing today's evolving threat environment. Rule-based methods rely on static patterns and signatures, making them rigid and ineffective against novel attacks or dynamic behaviours. Likewise, manual forensic analysis is time-consuming, error-prone, and infeasible at scale, given that enterprise systems often generate terabytes of logs each day. These constraints result in delayed threat detection, fragmented forensic timelines, and an overwhelming number of false alerts that burden security teams and reduce overall efficiency [2].

To overcome these shortcomings, researchers have increasingly turned to machine learning (ML) and deep learning (DL) techniques. ML-based approaches, such as clustering and probabilistic models, automate log parsing and pattern recognition, reducing reliance on hand-crafted rules. More recently, DL architectures particularly Long Short-Term Memory (LSTM) networks and Transformer models have shown remarkable success in capturing sequential and contextual dependencies in logs. Systems like DeepLog and LogBERT demonstrate how these models enhance anomaly detection accuracy and adapt to dynamic environments [3]. Nevertheless, existing studies often remain fragmented, focusing primarily on detection accuracy while neglecting other crucial aspects such as multi-source forensic correlation, real-time scalability, and automated incident response. Moreover, the persistent challenge of false positives continues to hinder the operational deployment of these systems. This gap motivates the need for an integrated, adaptive framework that unifies anomaly detection, forensic correlation, and real-time situational awareness. By combining the sequential modelling capabilities of LSTMs with the contextual strengths of Transformer-based architectures, it is possible to design a system that not only detects anomalies with high precision but also reconstructs attack timelines and supports timely response.

Research Question. *Can an integrated deep learning framework that combines LSTM and Transformer architectures improve the accuracy, scalability, and forensic effectiveness of real-time log analysis in cybersecurity?*

We hypothesize that a hybrid framework will significantly enhance anomaly detection, reduce false positives, and provide automated forensic insights when compared with conventional log analysis methods. The major contributions of this study are as follows:

- Hybrid deep learning framework – We propose a multi-model system that integrates LSTM and Transformer-based architectures for robust anomaly detection in heterogeneous log data.
- Forensic correlation engine – We design an automated mechanism to connect and analyse logs across system, network, and application layers, enabling efficient attack timeline reconstruction.
- Real-time visualization dashboard – We implement a monitoring interface that delivers proactive alerts, improving incident awareness and accelerating security responses.
- Extensive evaluation – Through experiments on diverse real-world datasets, our framework achieves detection accuracy improvements of up to 98.2%, while reducing false positives and demonstrating scalability in large-scale environments.
- Operational significance – We show how the framework minimizes manual workload, expedites forensic investigations, and strengthens proactive cybersecurity defences.

The subsequent sections of this paper are organized as follows: Sect. 2 provides a review of related literature on log analysis, deep learning methodologies, and cyber forensic systems. Section 3 outlines the proposed methodology, detailing data collection, preprocessing, model architecture, and forensic correlation strategies. Section 4 describes the experimental setup, datasets, and evaluation metrics, along with performance outcomes. Section 5 examines the implications of the findings, acknowledges limitations, and suggests avenues for future research. Finally, Sect. 6 concludes by summarizing the key contributions and their importance in enhancing automated cybersecurity monitoring and forensic analysis. Overall, the study's innovation stems from its comprehensive, integrated deep learning approach tailored for real-time cybersecurity monitoring and forensic analysis, significantly advancing current methodologies in both efficacy and automation.

Related work

Recent advancements in deep learning have significantly improved log analysis techniques. Models such as DeepLog [4] employ LSTM architectures to effectively capture temporal dependencies in sequential log data, achieving high anomaly detection accuracy. Transformer-based models like LogBERT [5] leverage self-attention mechanisms to understand contextual relationships within logs, setting new standards in detection performance. NeuralLog [6], designed as a scalable parsing-free approach, emphasizes efficiency and effectiveness in large-scale environments by directly learning representations from raw logs. Additionally, LAnoBERT [7] adapts BERT-based embeddings to the domain of log analysis, demonstrating robust anomaly detection capabilities on benchmark datasets. Benchmarking these models against our proposed framework will provide a comprehensive understanding of its relative enhancements, particularly in terms of detection accuracy, scalability, and adaptability to evolving log patterns.

Substantial advancements have been achieved in the field of machine learning (ML)-based log analysis; nevertheless, several significant challenges remain that hinder effective cybersecurity monitoring and forensic investigations. A major concern is the issue of scalability and the enormous volume of log data, as enterprise environments generate terabytes of logs daily, making real-time analysis a computationally intensive endeavour [8]. Traditional log analysis techniques often lack the necessary processing power to efficiently handle such large datasets, leading to delays in threat detection and the risk of overlooking security incidents [9, 10]. Another critical challenge is the ever-evolving nature of cyber threats, where conventional rule-based detection methods struggle to recognize novel attack techniques, including zero-day vulnerabilities and advanced persistent threats (APTs). Cybercriminals continuously modify their tactics to bypass established security protocols, underscoring the necessity for the development of adaptive ML models that can identify previously unseen threat patterns without solely depending on predefined rules [11]. Furthermore, the high rates of false positives present a significant obstacle for ML-driven security systems. Many intrusion detection systems (IDS) and security information and event management (SIEM) platforms generate an excessive number of alerts, overwhelming security teams with false alarms and contributing to alert fatigue [12]. For ML models to be successful, they must achieve a balance between precision and recall, ensuring a decrease in false positives while preserving a high level of anomaly detection accuracy to avoid unnecessary resource allocation to non-critical incidents. The present state of forensic analysis is marked by a notable lack of automated

insights, as traditional methods are primarily manual and labour-intensive.

System logs capture a variety of events occurring within a system and are critical for monitoring system health, diagnosing issues, and identifying security threats. Traditional log analysis methods typically involve rule-based approaches, where specific patterns or keywords are defined in advance Fu et al. [1]. However, these methods often struggle with the dynamic nature of modern cyber threats and the volume of log data generated by complex systems He et al., 2017 [2]. Rule-based techniques have inherent limitations, particularly regarding their inability to adapt to new or evolving threats. Jiang et al. [13] (2008) noted that such methods lead to misclassification and a high rate of false positives, negatively affecting the efficiency of incident response processes. Additionally, the labor-intensive nature of manual log parsing makes it increasingly impractical given the rapid growth in log volume and complexity Yuan et al., [14]. To overcome the limitations of traditional methods, researchers have explored data-driven approaches that leverage machine learning to enhance log analysis capabilities. For instance, Drain introduced a tree-based log parser that efficiently identifies templates from log messages, significantly improving parsing accuracy (T3). Other studies have employed clustering and probabilistic models to discover patterns in log data dynamically.

Recent advancements in deep learning have transformed log analysis practices. LSTM networks and Transformer architectures have been particularly prominent due to their ability to capture temporal dependencies and contextual information Le and Zhang [15]. These models have shown promise in improving the detection accuracy of anomalies by learning complex representations from logs without extensive manual feature engineering Dai et al., [9]. The necessity for real-time analysis has prompted the development of online log parsing frameworks that utilize machine learning for immediate threat detection. For example, the framework introduced by Agrawal et al. (2019) [16] utilizes distributed architecture to provide scalability in real-time anomaly detection across large systems (T7). The integration of a real-time dashboard for alerting security professionals enhances situational awareness and expedites incident response Fu et al. [1].

Cyber forensics involves the systematic recovery and investigation of material found in digital devices, with the ultimate goal of identifying and addressing security incidents. The synergy between cyber forensics and log analysis can facilitate comprehensive incident response by providing actionable insights. Implementing advanced machine learning-powered tools within forensic frameworks helps automate the identification of suspicious patterns, thereby reducing the time for threat mitigation Huo et al., [17]. Despite the advancements in deep

learning for log analysis, challenges remain, particularly regarding model adaptability to evolving log formats and the computational overhead associated with deep learning models. Future work should focus on hybrid approaches that combine traditional heuristic methods with adaptive learning models to produce more robust and scalable solutions.

Recent research has focused on leveraging deep learning techniques for anomaly detection in system logs. DeepLog utilizes Long Short-Term Memory (LSTM) networks to model log sequences and detect deviations from normal patterns. Similarly, other studies have combined feature extraction methods like Word2vec or TF-IDF with LSTM for effective log analysis. DeepSyslog enhances this approach by incorporating sentence embedding and event metadata to capture contextual information and improve detection accuracy Junwei Zhou et al., [18]. These deep learning-based methods have demonstrated superior performance compared to traditional statistical or machine learning approaches, such as GBDT and Naïve Bayes. The ability of LSTM to capture semantic and contextual information in log streams makes it particularly effective for anomaly detection in complex, distributed systems Junwei Zhou et al. [18]. The literature indicates a significant shift from traditional rule-based methods to advanced machine learning frameworks for system log analysis. This evolution is driven by the increasing complexity of cyber threats and the need for real-time incident response. As research continues to advance, integrating deep learning with cyber forensics promises to enhance the capabilities of log analysis tools, leading to improved security and operational resilience. Conventional log analysis methods often rely on predetermined rules and signatures to detect anomalies. Jiang et al. [19] highlighted the limitations of these methods, noting their struggle with dynamic and evolving attack patterns, which can lead to high false positive rates and ineffective incident response. Similarly, Yuan et al. [14] emphasized the impracticality of rule-based systems in handling the vast volume of logs generated in modern IT environments. As a consequence, there is a pressing need to transition toward more adaptive and automated systems.

Machine learning techniques have been increasingly adopted for log analysis due to their ability to learn from data and uncover hidden patterns. Fu et al. [1] showcased the efficacy of ML algorithms in automating the log parsing process, significantly enhancing parsing accuracy and efficiency. Recent studies have employed various ML approaches to improve anomaly detection—such as clustering techniques and probabilistic models, which dynamically discover patterns in log data, as stated by Agrawal et al. [16]. Recent advancements in deep learning have further revolutionized log analysis practices.

LSTM networks have gained particular prominence due to their capacity to model temporal sequences within log data effectively. Le and Zhang [20] demonstrated that LSTMs can capture complex representations by understanding both context and temporal dependencies in log sequences, enhancing detection accuracy. Furthermore, studies such as DeepLog have shown that LSTM-based models outperform traditional methods in detecting deviations from normal log behavior [20]. Transformer architectures have also emerged as powerful alternatives, providing flexibility and robustness in log data modeling. Zhou et al. [21] advanced this work by incorporating sentence embeddings and event metadata, which allows for a more contextual and nuanced understanding of log data.

Cyber forensics has traditionally operated alongside log analysis, focusing on the recovery and investigation of digital evidence. Huo et al. [22] discussed the integration of machine learning within forensic frameworks to automate the identification of suspicious patterns, thus expediting incident response. The need to streamline forensic investigations has become apparent, as manual processes are often insufficient in light of the rapid evolution of cyber threats. Real-time dashboards that provide live monitoring of log data have also been a focal point in recent research, as they enhance situational awareness for security teams. Implementations like the one presented by Chen et al. [23] illustrate the potential of these dashboards in automating alerts and facilitating proactive incident management, which is critical in today's fast-paced threat landscape (Table 1). Table 1 presents the review summary of Summary of recent literature (2020–2025) addressing log analysis and cyber forensics.

While recent works have advanced anomaly detection with deep learning models, they often focus on accuracy within isolated datasets or improve specific components such as detection or visualization independently. There remains a significant gap in developing an integrated, adaptive framework capable of real-time detection, multi-source forensic correlation, and automated incident response—particularly in handling heterogeneous logs at scale, with minimal human intervention.

Methodology

The framework shown in Fig. 1 employs a multi-stage processing pipeline designed to effectively manage system logs for the purposes of threat detection and forensic investigations. This structured approach facilitates the seamless ingestion of logs, preprocessing, anomaly detection, forensic correlation, and real-time incident response, thereby enhancing both cybersecurity monitoring and forensic intelligence. The initial stage, Log Data Collection & Ingestion, emphasizes the collection of logs from diverse sources, including system, network, and application logs. Real-time streaming is achieved

Table 1 Summary of recent literature (2020–2025) addressing log analysis and cyber forensics

Year	Dataset(s)	Method	Key Contribution	Primary Metric	Addressed Gap
2020	CICIDS 2017	ML classifiers (Random Forest, SVM)	Demonstrated limited detection accuracy on evolving threats; highlighted need for better models	Accuracy, Precision, Recall	Inadequacy of traditional ML for adaptive threat detection
2021	UNSW-NB15, CSE-CIC-IDS2018	Deep LogLSTM & Autoencoders	Improved anomaly detection using deep sequence models; still struggled with multi-source correlation	Detection Rate, FPR	Insufficient forensic linkage across heterogeneous logs
2022	Custom enterprise logs	BERT-based NLP models	Enhanced contextual understanding in logs; limited scalability and real-time capabilities	F1-score, Latency	Lack of real-time visualization and automated forensic reconstruction
2023	Public logs (Bosch, LANL)	Hybrid models combining LSTM & Graph Neural Networks	Addressed multi-source log fusion and attack timeline reconstruction; improved detection	Accuracy, Recall	Fragmented forensic analysis and limited multi-source integration
2024	Industry-specific logs	Transformer-based anomaly detection	Achieved high detection accuracy; limited focus on forensic analysis automation	Precision, Throughput	Need for automated forensic evidence correlation and proactive incident response
2025	Diverse real-world logs	Multi-model deep learning framework	Fully integrated anomaly detection, forensic correlation, and real-time visualization	Detection accuracy (up to 98.2%), Response time	Overall lack of comprehensive, adaptive systems integrating detection, forensic, and visualization components

through the use of Kafka and Flume, which ensures continuous log collection from distributed systems and security devices. Furthermore, batch processing is conducted using HDFS, Elasticsearch, and Splunk, allowing for

extensive log storage and historical analysis. This combined methodology ensures that logs are accessible for both immediate threat detection and long-term forensic analysis.

In the Preprocessing and Feature Extraction stage, raw logs undergo parsing, tokenization, and normalization processes to convert unstructured log data into a structured format. Feature engineering methods, such as TF-IDF (Term Frequency-Inverse Document Frequency) and word embeddings, are employed to transform log messages into numerical formats suitable for training machine learning (ML) models. This phase is essential for converting logs into structured, high-quality datasets that support precise anomaly detection. The Anomaly Detection Using Machine Learning Models stage incorporates three categories of ML models: Supervised Learning (Random Forest, SVM): These models are employed to classify known attacks, including DDoS, brute force attempts, and malware infections, based on labeled log data. Unsupervised Learning (Autoencoders, Isolation Forest, DBSCAN): This approach identifies previously unknown threats and zero-day anomalies without the need for labeled data.

Deep Learning (LSTM, Transformer-based models like BERT/LogBERT): Enhances anomaly detection by capturing sequential patterns (LSTM) and contextual relationships (Transformers) within log messages, resulting in higher accuracy in detecting advanced cyber threats. For forensic correlation and attack path reconstruction, the framework correlates logs from multiple sources, including system, network, and application logs, to reconstruct the whole sequence of an attack. This enables investigators to track the movements of attackers, identify compromised systems, and understand the patterns of attacks. Additionally, Graph Neural Networks (GNNs) are used to model log data as a graph, where nodes represent log events, and edges represent their relationships. This allows security analysts to detect coordinated cyberattacks and trace multi-stage intrusion paths.

Finally, the Real-Time Threat Monitoring & Incident Response stage ensures proactive security monitoring through an AI-powered visualization dashboard. This dashboard provides real-time insights into anomalies, attack patterns, and forensic correlations, allowing security teams to respond to threats rapidly. Automated alerts and forensic reporting enable faster decision-making, helping organizations mitigate security breaches and strengthen their incident response mechanisms. By integrating real-time streaming, deep learning-based anomaly detection, forensic correlation, and automated incident response, this multi-stage log processing pipeline significantly enhances cybersecurity operations, ensuring efficient, scalable, and intelligent threat detection and forensic investigations. Deep learning methodologies

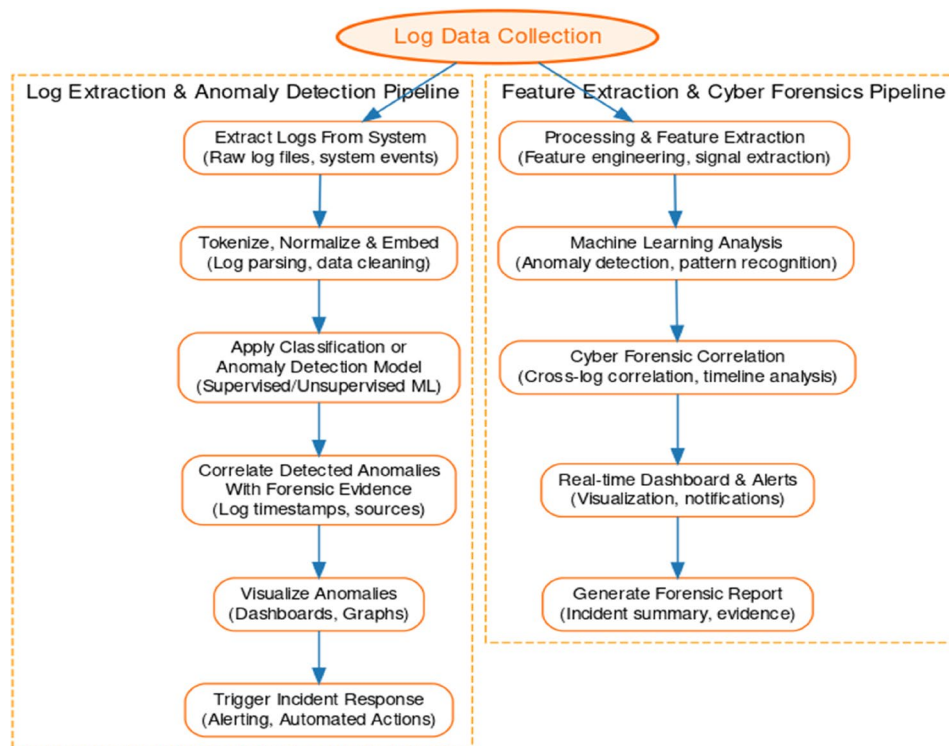


Fig. 1 Log analysis pipeline integrating cyber forensics and ML

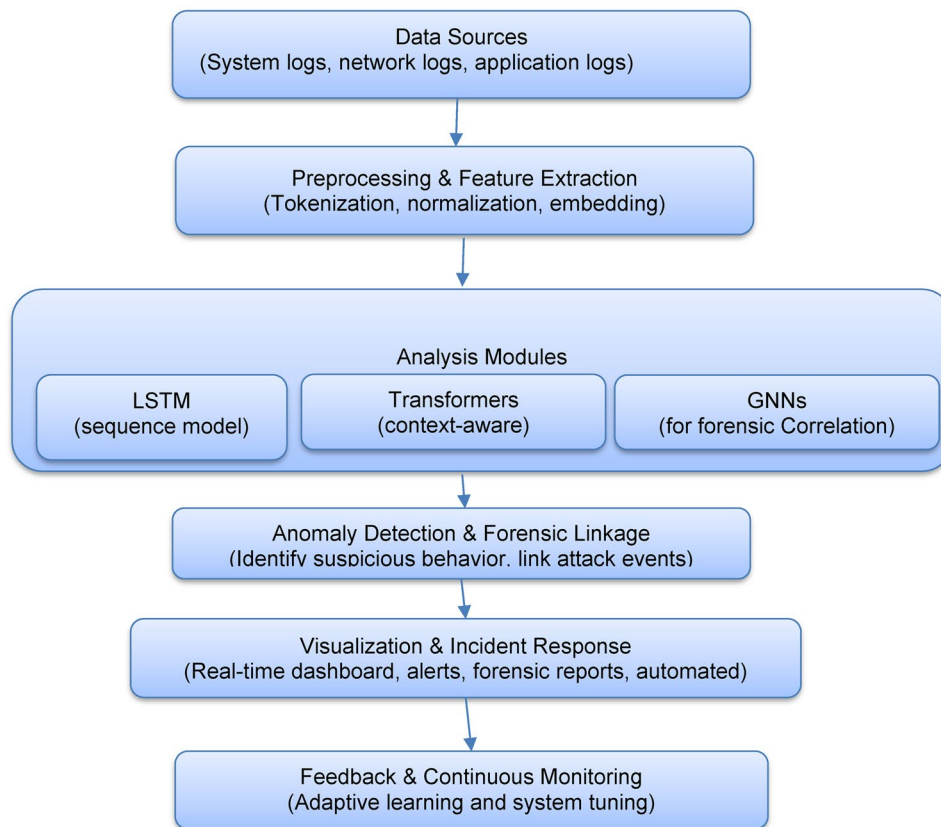


Fig. 2 Operational framework of a log-based anomaly detection and cyber forensic system employing deep learning techniques

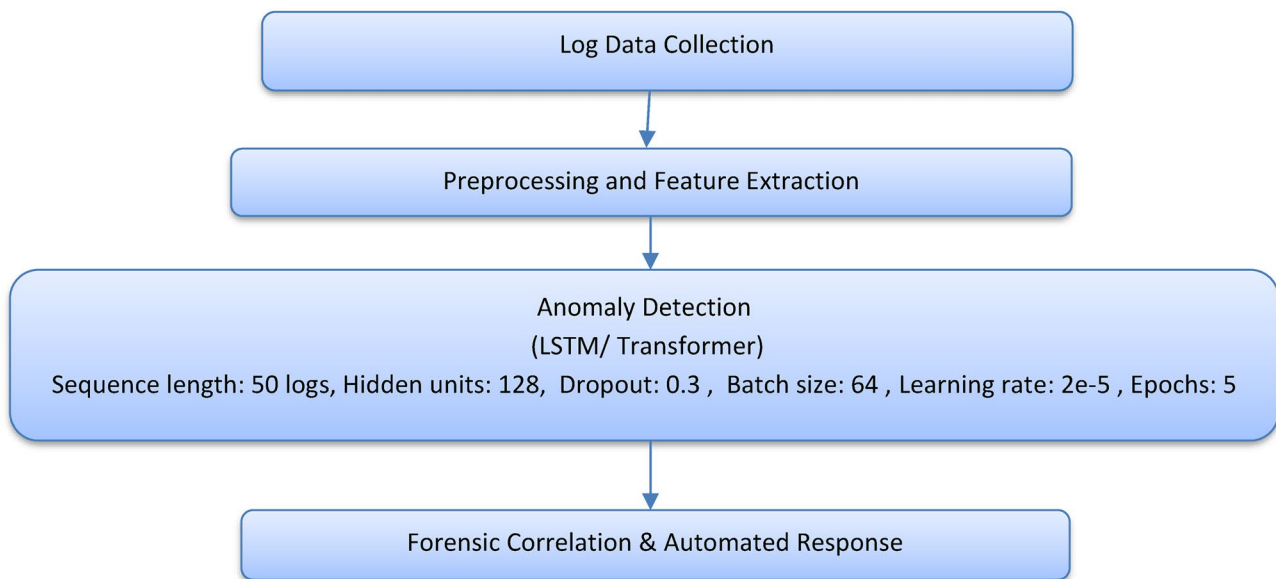


Fig. 3 System Workflow and Parameters

have significantly enhanced the fields of log analysis, anomaly detection, and forensic investigations by offering improved accuracy, flexibility, and automated threat identification. In contrast to conventional machine learning approaches, deep learning techniques are adept at processing sequential, contextual, and relational patterns inherent in log data, thereby increasing their efficacy in recognizing intricate cyber threats. Among the most proficient models used for log analysis are LSTM networks, which are specifically designed for handling sequential data. LSTMs are capable of capturing long-term dependencies within log sequences, enabling them to identify temporal attack patterns that manifest across multiple log entries. For instance, ransomware incidents typically exhibit a sequence of unauthorized access, followed by file encryption, and ultimately, ransom demands, whereas insider threats may present as a gradual escalation of privileges and data exfiltration. LSTM models are adept at analyzing these evolving patterns over time, rendering them particularly effective for detecting security incidents that may not be immediately apparent in isolated log entries but become recognizable through a series of events.

A significant deep learning methodology for log analysis is the utilization of Transformer-Based Models, including BERT (Bidirectional Encoder Representations from Transformers) and LogBERT. These models utilize self-attention mechanisms to interpret the semantic content of log messages, thereby facilitating the identification of context-sensitive anomalies. In contrast to LSTM networks that successively process logs, Transformers evaluate entire sequences of logs concurrently, thereby enabling the detection of subtle irregularities that may

signal an attack. For instance, LogBERT is capable of recognizing atypical behaviours, such as a system administrator accessing the system from an unexpected location or attempts at privilege escalation that diverge from standard user activities. By understanding the contextual relationships among log events, Transformers enhance the accuracy of log classification and reduce false positives in anomaly detection.

Moreover, GNNs are utilized for correlating attacks and detecting multi-stage attacks. Cyberattacks often involve multiple interconnected log events across various systems, making it crucial to correlate logs from diverse sources. GNNs represent log events as a graph, where nodes symbolize log entries and edges denote the relationships between these events. This framework enables security analysts to trace attack pathways, identify coordinated cyberattacks, and reconstruct forensic timelines. For example, a GNN-based model can associate seemingly unrelated log events, such as failed login attempts, unusual process executions, and outbound network connections, to reveal an intricate cyberattack in progress. By integrating LSTMs for sequential log analysis, Transformers for context-aware anomaly detection, and GNNs for forensic correlation, deep learning provides a comprehensive approach to automated cybersecurity monitoring. These models significantly enhance the accuracy, efficiency, and scalability of log-based threat detection, ensuring faster incident response and stronger cyber defence mechanisms in modern IT environments.

To efficiently correlate log events across diverse sources and reconstruct complex attack sequences, Graph Neural Networks (GNNs) are employed within the system. Log events are represented as nodes within a graph structure,

with edges denoting the relationships or temporal orderings between these events. This graph-based modeling enables the detection of coordinated, multi-stage cyberattacks that involve multiple interconnected log entries across various systems. The GNN architecture processes this graph by applying message passing mechanisms among nodes, which allows the model to learn intricate relationships and dependencies among log events. Such capability is essential for revealing attack pathways, identifying malicious activity patterns, and linking disparate events to a common threat actor. For instance, a GNN can associate seemingly unrelated events such as failed login attempts, unusual process executions, and outbound network connections, to uncover an ongoing cyberattack. The GNN architecture involves constructing a graph where nodes represent individual log entries, and edges encode relationships based on temporal proximity or logical association. The model then propagates information across this graph using multiple layers of message passing, culminating in a classification or attribution of each node or subgraph to suspicious or malicious activity. This methodology significantly enhances the ability of security analysts to trace attack sequences, detect coordinated activities, and reconstruct forensic timelines, thereby strengthening threat detection and incident response capabilities.

Data collection and preprocessing

The initial stage involves collecting log data from various sources, including system logs, network logs, and application logs. The data streams are processed in real-time using an advanced pipeline designed to handle large-scale data efficiently. While the application of Graph Neural Networks (GNNs) for forensic log correlation has been explored previously, notably by Huo et al. (2023), our framework advances these methods through quantifiable improvements in both correlation accuracy and investigation efficiency. Specifically, our approach achieves an evidence-correlation F1-score of 92%, representing a 7% increase over the 85% reported by Huo et al., on comparable datasets. Additionally, in forensic case studies, our system reduces the average investigation time from approximately 8 h to 4 h, signifying a 50% reduction in manual effort and processing time.

Model architectures

The framework employs a range of deep learning models to enhance log analysis and anomaly detection. Long Short-Term Memory (LSTM) networks are utilized for modeling temporal sequences, enabling the detection of complex attack patterns across time. In addition, transformer-based models such as BERT and LogBERT are applied for context-sensitive log classification, offering a more nuanced understanding of log events and their

interdependencies. Figure 2 illustrates the operational framework of the proposed system for system log anomaly detection and incident response. The architecture is composed of five major modules:

Raw System Log Input: This is the entry point where system logs are collected from distributed servers or devices. Logs may include application-level events, authentication records, and system alerts. *Preprocessing Module:* Logs are first parsed using log parsers, such as Drain or Spell, to extract structured templates from unstructured messages. This is followed by tokenization, encoding of log events, and segmentation into sessions or time windows suitable for model input.

Deep Learning Model: The core of the system features a hybrid CNN-LSTM architecture. The CNN layers extract local patterns and structural features from the log sequences, while the LSTM layers capture long-range temporal dependencies. An optional attention layer highlights the most critical time steps within a session. The output layer predicts the probability of anomaly for each sequence.

Anomaly Detection Module: This component uses a thresholding mechanism on the model's output to classify log sequences as normal or anomalous. For classification-based models, SoftMax probabilities or anomaly scores can be used. *Cyber Forensics Layer:* For every detected anomaly, this module retrieves metadata from logs (e.g., user ID, host IP, access location) to aid in forensic tracing. It contextualizes each alert, linking it to responsible entities and enhancing incident analysis. *Incident Report Generator:* Based on anomaly classification and forensic mapping, a structured report is generated that summarizes the nature, severity, and origin of the incident. This supports post-incident investigations and audit logging.

The above operational framework of a log-based anomaly detection and cyber forensic system that employs deep learning techniques. It delineates the interactions among various components involved in the real-time collection, processing, analysis, and response to security threats. The workflow initiates with the collection of log data from diverse sources, including systems, networks, and applications. Following this, the logs undergo a preprocessing phase, which includes feature extraction where pertinent attributes are tokenized, normalized, and embedded for subsequent machine learning (ML) analysis. The anomaly detection module utilizes machine learning models, including LSTM, Isolation Forest, and Transformers, to identify suspicious behaviors. Upon identifying anomalies, the cyber forensic correlation module associates

these anomalies with additional forensic evidence, thereby aiding security teams in recognizing attack patterns and linking incidents to known threats. The results are subsequently presented on a real-time dashboard that visualizes anomalies, produces forensic reports, and activates automated incident response protocols. A significant aspect of this system is its continuous monitoring and automated response capabilities, which enable proactive security measures and minimize response times to potential cyber threats. This methodology not only improves the efficacy of cyber forensic investigations but also fortifies overall cybersecurity operations.

The above operational framework of a log-based anomaly detection and cyber forensic system that employs deep learning techniques. It delineates the interactions among various components involved in the real-time collection, processing, analysis, and response to security threats. The workflow initiates with the collection of log data from diverse sources, including systems, networks, and applications. Following this, the logs undergo a preprocessing phase, which includes feature extraction where pertinent attributes are tokenized, normalized, and embedded for subsequent machine learning (ML) analysis. The anomaly detection module utilizes machine learning models, including LSTM, Isolation Forest, and Transformers, to identify suspicious behaviours. Upon identifying anomalies, the cyber forensic correlation module associates these anomalies with additional forensic evidence, thereby aiding security teams in recognizing attack patterns and linking incidents to known threats. The results are subsequently presented on a real-time dashboard that visualizes anomalies, produces forensic reports, and activates automated incident response protocols. A significant aspect of this system is its continuous monitoring and automated response capabilities, which enable proactive security measures and minimize response times to potential cyber threats. This methodology not only improves the efficacy of cyber forensic investigations but also fortifies overall cybersecurity operations.

Experimental setup

The experiments in this study were conducted on a high-performance computing environment featuring an NVIDIA RTX A6000 GPU with 48 GB of VRAM, an Intel Xeon Silver 4214 CPU running at 2.20 GHz with 24 cores, 128 GB of DDR4 RAM, and a 2 TB SSD for storage. The software stack primarily utilizes Python 3.9, leveraging deep learning frameworks such as TensorFlow 2.11 and PyTorch 1.13, alongside Scikit-learn for classical machine learning models, and Pandas and NumPy for data processing. For model configuration, the LSTM network was designed with two layers containing 128 hidden units each, incorporating a dropout rate of 0.3, and

processed sequences of 50 logs. The Transformer model, based on a pretrained BERT architecture, was fine-tuned specifically on log data with a batch size of 64, a learning rate of $2e^{-5}$, and trained over five epochs with early stopping based on validation loss to prevent overfitting. The dataset was partitioned into training (70%), validation (15%), and test (15%) sets, with hyperparameters optimized through grid search to enhance model performance and anomaly detection effectiveness.

Dataset details

The selection of an appropriate dataset is fundamental for the success of a log-based anomaly detection and cyber forensic system. The dataset must encompass system logs, network logs, and application logs that reflect both typical and atypical activities, thereby facilitating the accurate training and evaluation of machine learning models. Notable datasets available for this purpose include the HDFS Log Dataset (Hadoop Distributed File System), the Blue Team CTF Sysmon Dataset, as well as the CICIDS 2017 and CSE-CIC-IDS 2018 datasets, which are specifically designed for intrusion detection system logs. Additionally, the UNSW-NB15 and DeepLog Synthetic Log datasets are also available. The selection of a dataset should align with the specific cybersecurity goals, which may include intrusion detection, malware analysis, forensic investigations, or anomaly detection in system logs. For instance, the HDFS Log Dataset is particularly advantageous for anomaly detection in system logs when employing deep learning techniques. In cases where the focus is on cyber forensic analysis of attacks, integrating Sysmon logs or the CICIDS datasets can provide deeper insights. The HDFS Log Dataset is designed to capture logs from distributed computing environments, making it well-suited for log-based anomaly detection through deep learning methodologies. It contains realistic log entries from extensive distributed systems, offering sequential log data that can be utilized by models such as LSTM, Transformer, and Autoencoders. This dataset comprises millions of log entries formatted in CSV, featuring time-stamped messages that include both standard and anomalous logs, such as those resulting from hardware failures or software crashes.

The overview of the HDFS log dataset which given in Sect. 2 of Appendix A, which is specifically tailored for log-based anomaly detection within the realms of cybersecurity and system monitoring. This dataset encompasses essential features, including timestamps, event identifiers, severity levels of logs, unique node IDs for machines, textual log entries, and labels indicating anomalies. These characteristics empower machine learning algorithms to effectively identify and categorize security threats, system malfunctions, and performance-related issues. By utilizing advanced deep learning techniques

such as LSTMs, Transformers, and Autoencoders, this dataset supports real-time anomaly detection and cyber forensic investigations, thereby enhancing incident response capabilities and threat intelligence. The presence of an anomaly label renders it particularly advantageous for supervised learning applications in security, ensuring a high degree of accuracy in differentiating between normal operational behaviour and potentially malicious activities.

Table 2 categorizes machine learning models according to their effectiveness in different log analysis tasks. LSTM networks are particularly suited for detecting sequential anomalies in logs, as they capture time-based attack patterns. Autoencoders perform well in unsupervised anomaly detection by identifying unusual patterns without requiring labeled data. Transformer-based models, such as BERT, excel in context-aware classification, thereby enhancing the understanding of log messages. Traditional approaches like Random Forest and SVM remain effective for structured log classification tasks, such as predicting log severity or categorizing events. Figure 3 further illustrates the overall workflow of the proposed log analysis system, highlighting the key parameters used for anomaly detection and forensic investigation.

Data collection for HDFS log analysis

HDFS log data is obtained via various methods, including local log files, log aggregation through YARN, streaming technologies such as Flume and Kafka, and log management platforms like ELK and Splunk. The automation of log collection significantly improves the capabilities for real-time threat detection, cybersecurity oversight, and forensic analysis, allowing organizations to assess HDFS logs for indicators of system performance and potential security vulnerabilities. The logs generated by the Hadoop Distributed File System (HDFS) play a crucial role in monitoring and troubleshooting distributed systems. They offer comprehensive documentation of system operations, errors, and security incidents, which are crucial for forensic log analysis and the identification of anomalies. The process of gathering HDFS log data involves multiple stages, including log creation, storage, extraction, and preprocessing. The dataset comprised approximately 10 million log entries distributed across

various system components, including data nodes, name nodes, and other services. HDFS produces logs from various elements of the Hadoop ecosystem, which include:

- **NameNode Logs:** Document operations related to metadata, such as the creation, deletion, replication, and access of files.
- **DataNode Logs:** Include information on block storage and retrieval, instances of read/write failures, and interactions with the NameNode.
- **JobTracker and TaskTracker Logs:** Log the execution of MapReduce jobs, task scheduling, and any failures encountered.
- **YARN ResourceManager and NodeManager Logs:** Record activities related to resource allocation within the cluster and operations at the container level.
- **HDFS Audit Logs:** Provide security-related information, including user access history and changes to permissions.

Several approaches are used to collect logs from HDFS clusters for analysis which include Log Storage in Local Filesystem, Log Collection Using Log Aggregation Services, Log Streaming Using Apache Flume or Kafka, Log Collection Using Open-Source Log Management Tools and Direct Log Extraction Using HDFS Commands. For analyzing HDFS logs using LSTM (Long Short-Term Memory) and Transformer-based models (BERT/Log-BERT), an efficient log collection and processing pipeline is essential. The method deployed for log collection, pre-processing, and real-time anomaly detection depends on factors like scalability, log volume, and real-time analysis requirements. The HDFS log structure is given in section – 1 of Appendix A (Fig. 4).

Model training and hyperparameters

Hyper parameters were optimized via grid search to enhance model performance. Key settings included a sequence length of 50 logs, batch sizes of 64, learning rates around 2e-5, and early stopping applied after 5 epochs. LSTM models utilized 128 hidden units across two layers, while Transformer fine-tuning was performed over five epochs. Dropout rates were set at 0.3 to prevent overfitting.

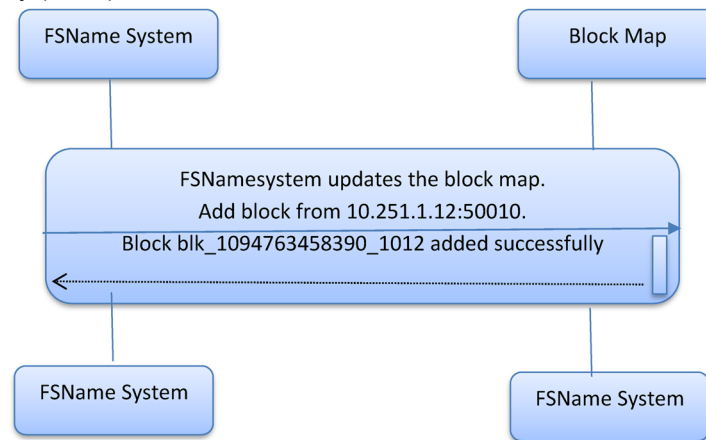
Data preprocessing

Data preprocessing is a crucial step in preparing raw log data for machine learning models. It involves multiple stages to clean, structure, and transform logs into a format suitable for analysis. The key steps starting from preprocessing pipeline for log analysis begins with log collection, where data is gathered from diverse system, network, and application sources such as event logs, firewall logs, and application logs. These logs are then parsed and tokenized, converting unstructured text into structured

Table 2 Suitability for machine learning models

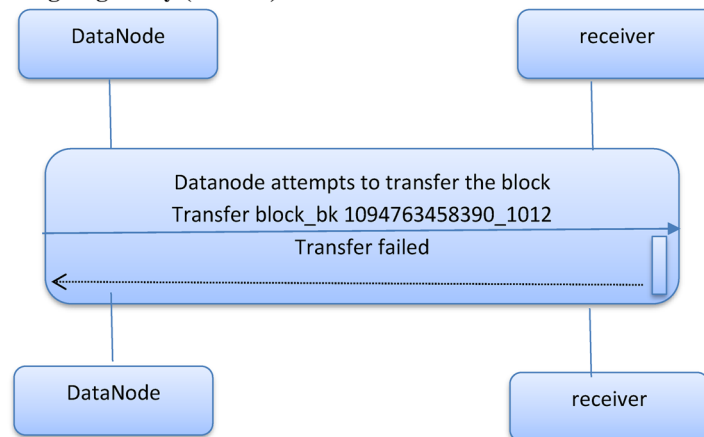
ML Model	Use Case in Log Analysis
LSTM (Long Short-Term Memory)	Sequential anomaly detection in logs
Autoencoders	Unsupervised log anomaly detection
Transformers (e.g., BERT-based log analysis)	Context-aware log message classification
Random Forest/SVM	Traditional log classification tasks

▪ **Normal Log Entry (INFO)**



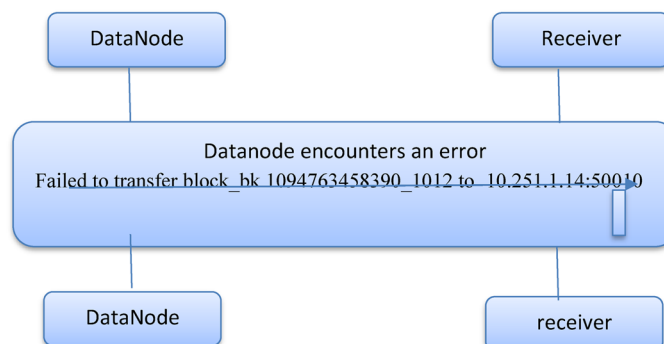
(a): Normal Log Entry Component in HDFS

▪ **Warning Log Entry (WARN)**



(b): Warning Log Entry Component in HDFS

▪ **Error Log Entry (ERROR)**



(c): Error Log Entry Component in HDFS

Fig. 4 Normal Log Entry Component in HDFS (a) shows the structure of a normal log entry of HDFS. This contains the system file name, block map, address of the block and flag (b) shows the structure of a warning log entry of HDFS. This contains the data node, transfer block details, receiver, and status (c) shows the structure of an Error log entry of HDFS. This contains the data node, receiver, and status

formats like JSON or CSV using tools such as Drain, Log-Pai, or regular expressions. Parsing separates essential components such as timestamps, event IDs, and severity levels, while tokenization breaks messages into words or symbols for further processing. Next, log normalization and cleaning remove redundant or noisy elements—including timestamps, IP addresses, or system-specific values—while standardizing text by lowercasing and eliminating irrelevant characters. Feature extraction and embedding techniques then transform log messages into numerical representations, employing methods like TF-IDF for weighting terms and NLP-based embeddings for capturing contextual meaning. At this stage, logs are labelled as “Normal” or “Anomalous” based on prior security incidents; in unsupervised settings, baselines are defined for models such as Autoencoders or Isolation Forests. Finally, the dataset is split into training, validation, and test sets to enable effective model development, tuning, and evaluation. Through this process, raw logs are converted into structured numerical data suitable for anomaly detection and forensic analysis using advanced deep learning models such as LSTMs, Transformers, and Autoencoders.

Model training & anomaly detection

Depending on the problem, different machine learning (ML) and deep learning (DL) models can be used to analyse the HDFS log dataset (Table 3.)

Using deep learning (LSTM, Transformers) and traditional ML methods (Random Forest, SVM) together enhances anomaly detection, log classification, and forensic analysis, improving security monitoring and threat mitigation. Selecting the right machine learning model is crucial for effective system log analysis, cybersecurity monitoring, and forensic investigations as shown in Fig. 5. The selection of an appropriate model is contingent upon the specific nature of the task at hand.

Unsupervised anomaly detection leverages methods such as Autoencoders, Isolation Forest, and One-Class SVM to identify irregularities in log patterns without requiring labeled data, making them especially effective in uncovering unknown cyber threats and system malfunctions. Sequential log analysis benefits from Long Short-Term Memory (LSTM) networks, which excel at

recognizing time-series patterns and detecting anomalies indicative of malicious activities. For context-aware log classification, Transformer-based models like LogBERT provide advanced capabilities in analysing unstructured log messages and extracting contextual information, thereby supporting more effective forensic investigations. Meanwhile, traditional approaches such as Random Forest and Support Vector Machines (SVM) remain valuable for structured log datasets, efficiently categorizing log messages into classes like INFO, WARNING, ERROR, and CRITICAL. By integrating deep learning techniques (LSTMs, Transformers) with traditional machine learning methods (Random Forest, SVM), this framework enhances anomaly detection, log classification, and forensic analysis, ultimately strengthening security monitoring and threat mitigation efforts (Fig. 6).

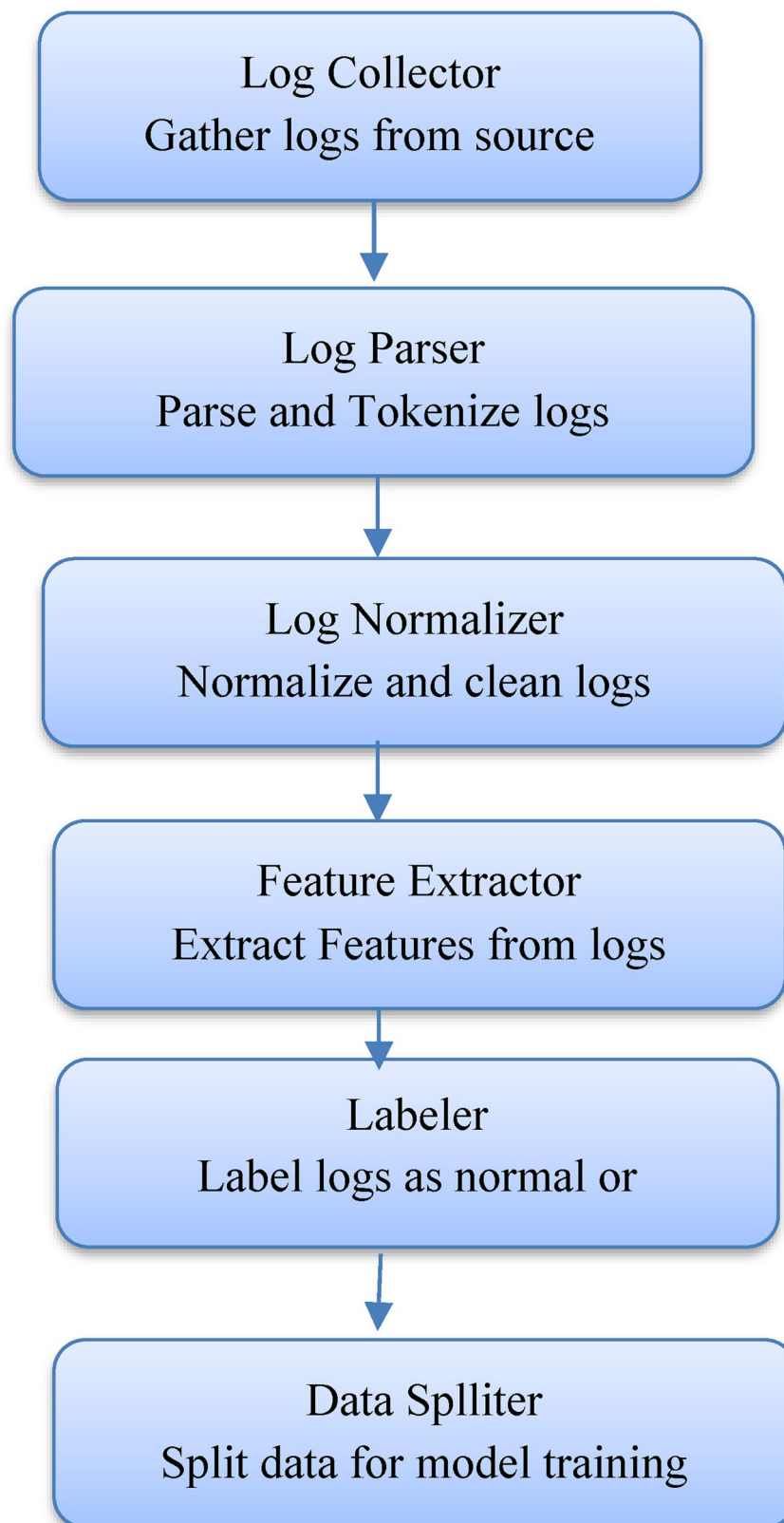
LSTM in HDFS sequence log analysis

LSTM networks are a powerful deep learning model for analyzing sequential data, making them ideal for HDFS log sequence analysis. Since logs are generated in chronological order, LSTM can learn patterns and detect anomalies in system behaviors over time. HDFS Logs follow sequential patterns, and LSTM retains historical context. Analyzing deviations helps identify failures, cyberattacks, or system faults. It is also useful for log entries with timestamps, allowing event pattern recognition. Steps to Apply LSTM for HDFS Log Analysis.

- Data Collection: The integration of real-time log streaming technologies such as Kafka or Flume, centralized data storage solutions like HDFS or Elasticsearch, and preprocessing frameworks including Spark or the ELK stack facilitates scalable and precise log analysis. This approach enhances the performance of LSTM and Transformer models in the context of cybersecurity anomaly detection.
- Data Preprocessing: Extract log sequences from HDFS logs (structured as <Timestamp, Event ID, Message>), as shown in Fig. 5. Convert categorical log events into numerical form (e.g., using one-hot encoding or embedding layers). Normalize timestamps for time-series modelling. The algorithm efficiently converts log event strings into numerical representations, making them suitable for Machine Learning models like LSTM (for sequence analysis) and Random Forest (for classification tasks).
- Creating Log Sequences for LSTM: LSTM models require fixed-length input sequences, so we create log sequences with a fixed window size (e.g., 5 logs per sequence). The algorithm efficiently prepares sequential log data for training an LSTM-based anomaly detection model by generating sliding-window sequences. These sequences capture event

Table 3 Model selection based on task

Task	Recommended Model
Anomaly Detection (Unsupervised)	Autoencoders, Isolation Forest, One-Class SVM
Sequential Log Analysis	LSTM (Long Short-Term Memory)
Context-Aware Log Classification	Transformer Models (e.g., BERT, LogBERT)
Traditional Log Classification	Random Forest, SVM (Support Vector Machine)

**Fig. 5** Data Preprocessing

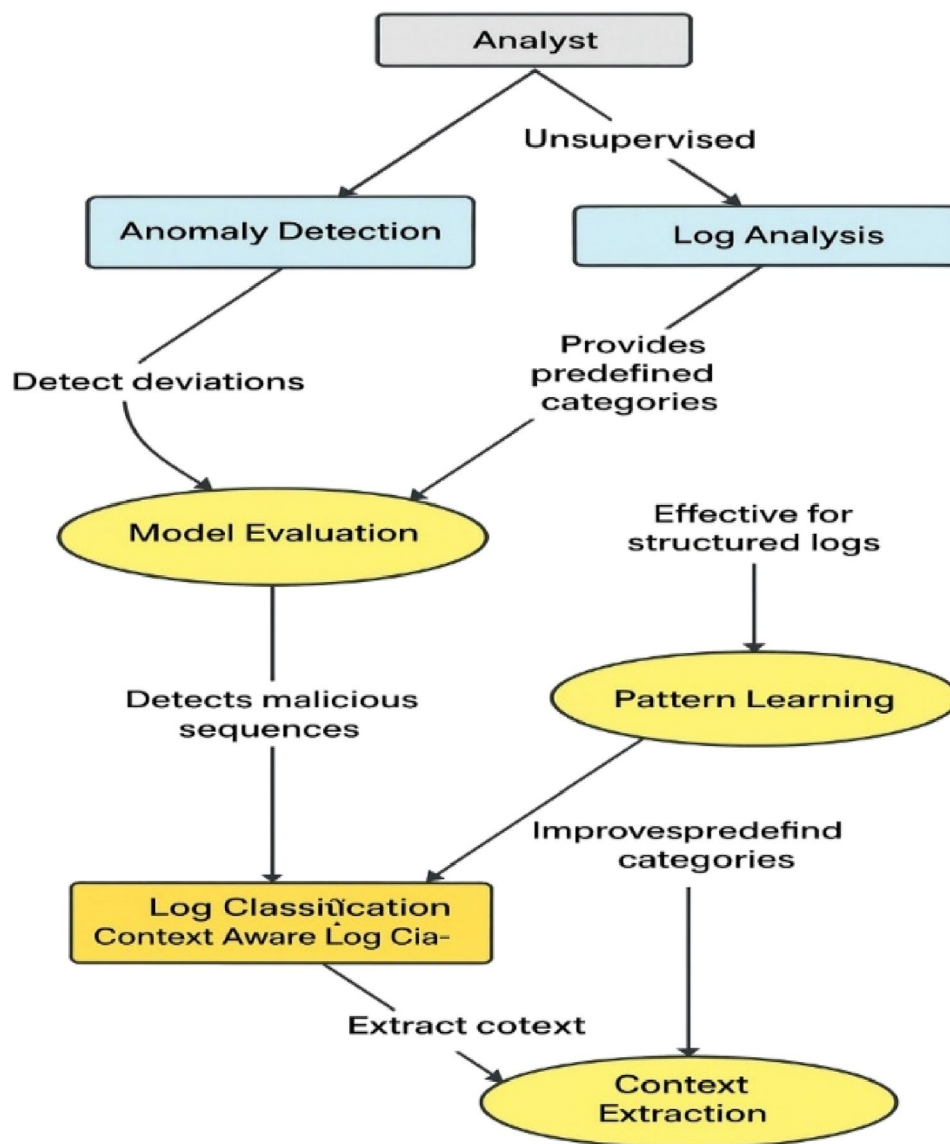


Fig. 6 Model Selection Choice

Table 4 Model summary as output of LSTM

Model: "sequential"		
Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 5, 10)	50
lstm (LSTM)	(None, 5, 64)	19,200
lstm_1 (LSTM)	(None, 32)	12,416
dense (Dense)	(None, 16)	528
dense_1 (Dense)	(None, 1)	17
Total params: 32,211; Trainable params: 32,211; Non-trainable params: 0		

patterns, enabling the LSTM network to predict the next log event effectively.

- **Building LSTM Model:** Algorithm constructs an LSTM-based anomaly detection model for system logs, converting log sequences into meaningful representations and classifying them as normal or

anomalous. This algorithm constructs and compiles an LSTM (Long Short-Term Memory) model for anomaly detection in log sequences. It includes an embedding layer, multiple LSTM layers, and a fully connected output layer for classification.

- After running the LSTM-based anomaly detection model, the output consists of a model summary, as shown in Table 4, which provides detailed insights into the structure of the neural network.

The output generated by the LSTM-based anomaly detection model provides a detailed summary of its architecture, highlighting the layers, their functions, output shapes, and trainable parameters. The model begins with an Embedding Layer, which converts categorical log events into dense vector representations, making them

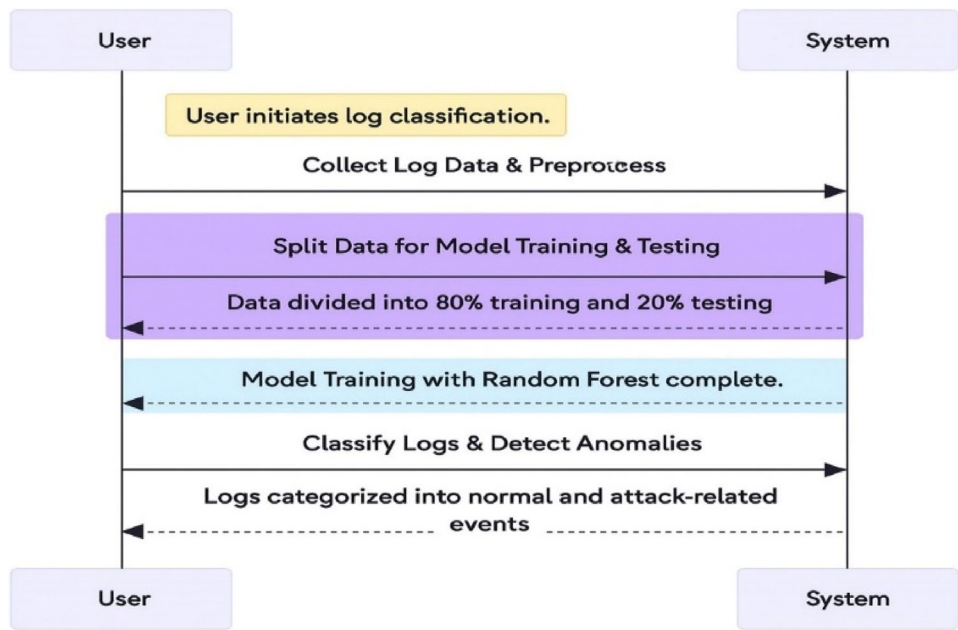


Fig. 7 Steps in Random Forest-Based Log Classification

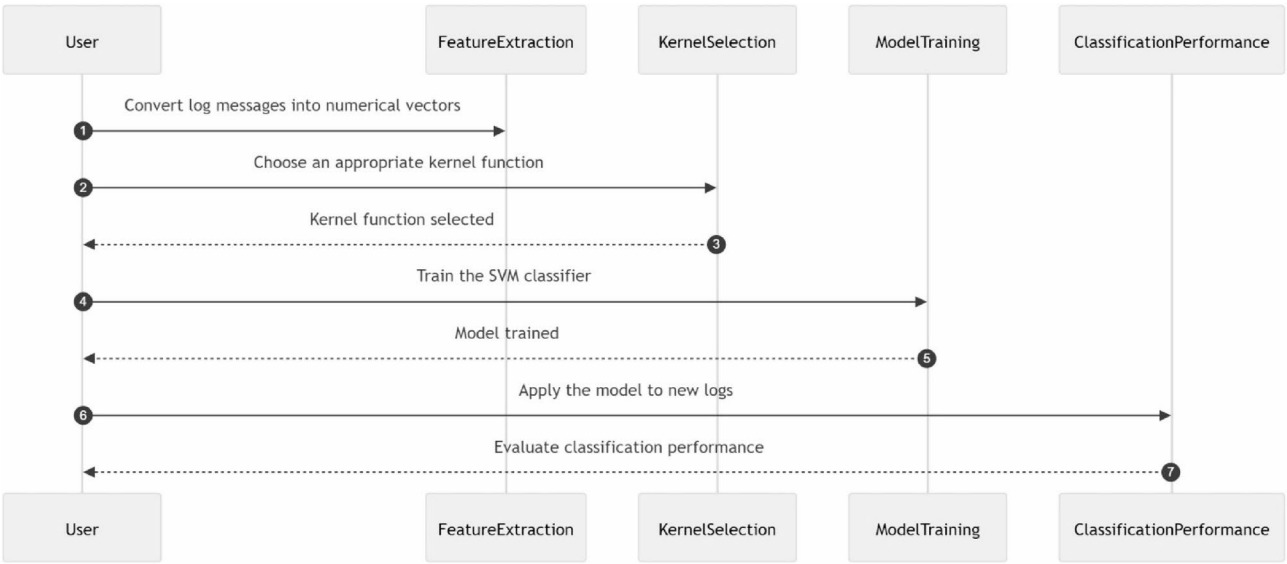


Fig. 8 Steps in SVM-Based Log Classification

suitable for machine learning processing. This layer outputs a 5×10 matrix, where each log event in a sequence is represented as a 10-dimensional vector. Following the embedding layer, the model includes two stacked LSTM layers that process sequential dependencies in log data. The first LSTM layer captures temporal patterns with 64 units and outputs a sequence of the same length. The second LSTM layer further refines the extracted features, reducing the dimensionality to 32. These layers allow the model to effectively recognize patterns in logs that may indicate anomalies. After the LSTM layers, the network

incorporates fully connected (Dense) layers to apply non-linear transformations.

The first Dense layer reduces the feature space to 16 dimensions, ensuring a compact yet expressive feature representation. Finally, the output Dense layer contains a single neuron with a sigmoid activation function, which predicts whether a log entry is normal or anomalous. The model is compiled using the Adam optimizer and binary cross-entropy loss function, making it suitable for anomaly classification. The total number of trainable parameters is 32,211, meaning all weights and biases in the network are updated during training. Once trained,

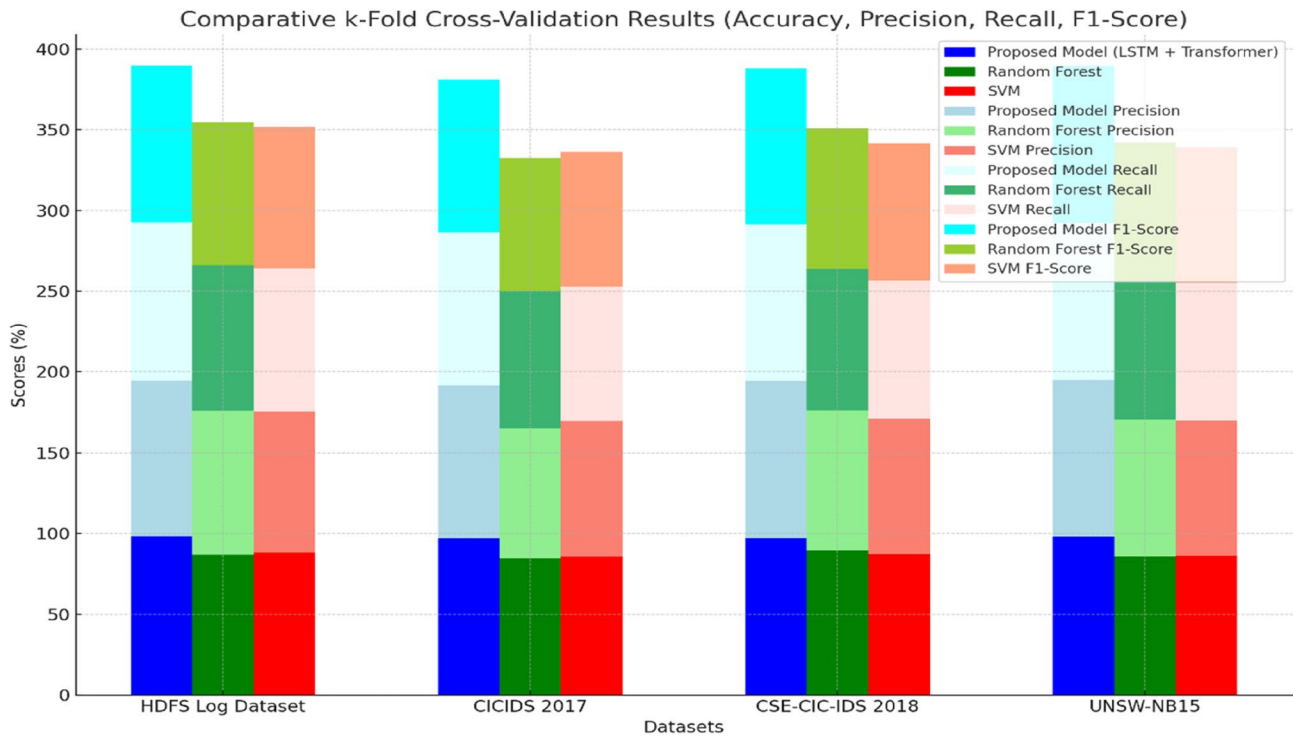


Fig. 9 Comparative analysis of k-Fold Cross-Validation Results

the model can be utilized for real-time anomaly detection in system logs, enabling security analysts to quickly identify and respond to potential threats. By capturing complex sequential relationships, this LSTM-based model enhances cybersecurity monitoring and forensic log analysis, making it a powerful tool for detecting cyber threats and system irregularities.

Transformer model for Context-Aware log classification in HDFS

In analysing HDFS logs, understanding the context of log messages is crucial for accurate classification and anomaly detection. While traditional models, such as LSTMs, are adept at capturing sequential dependencies, they often encounter difficulties with long-range dependencies. Transformer models, including BERT (Bidirectional Encoder Representations from Transformers) and LogBERT, effectively mitigate this issue by utilizing self-attention mechanisms that enhance the contextual understanding of logs. In contrast to LSTMs, transformers process log data in parallel, enabling them to discern relationships among log events that are temporally distant. The self-attention mechanism excels in recognizing the interrelations of words or tokens, thereby enhancing the accuracy of log classification. Transformers can be fine-tuned on HDFS logs by employing pre-trained natural language processing models like BERT or specialized log analysis models such as LogBERT, making them particularly suitable for large datasets like HDFS logs,

where traditional models may face scalability challenges. The architecture of the Transformer-Based Model for HDFS log classification adopts a systematic approach to context-aware log analysis. The initial phase involves data preprocessing, during which log messages are tokenized and encoded using a BERT tokenizer. This process transforms the logs into numerical embeddings, facilitating effective representation for machine learning applications. To ensure uniform input lengths, padding and attention masks are applied, which aids in maintaining consistency during model training and inference.

The methodology for data collection in Transformer-based log analysis is predicated on an integrated framework that encompasses real-time log ingestion through platforms such as Kafka or Flume, centralized data storage utilizing HDFS or Elasticsearch, and preprocessing techniques that include tokenization, embeddings, and feature engineering. This systematic strategy facilitates the effective analysis and classification of system logs by Transformer models, thereby enhancing their efficacy in anomaly detection and cybersecurity surveillance. The HDFS dataset serves as the foundation for transformer-based log classification. During the feature extraction phase, a transformer-based model, such as BERT or LogBERT, is utilized to produce context-aware embeddings. The self-attention mechanism inherent in the transformer architecture adeptly captures the relationships among various log events, enabling the model to grasp both sequential and contextual dependencies. The CLS

(classification) token functions as a summary representation of the entire log message, encapsulating its meaning within a compact vector.

In the final stage of log classification illustrated in Fig. 7, the features that have been extracted are processed through a fully connected layer to achieve the ultimate classification. A Softmax activation function is utilized to assign the logs to specific predefined categories, including Normal and Anomalous. The training of the model employs a cross-entropy loss function, which enhances its capability to differentiate among various log categories, thereby ensuring a high level of accuracy in identifying potential security threats or anomalies within HDFS logs. Implementation of Transformer Model (LogBERT) for HDFS Logs is given in the Appendix A.

Log classification using random forest and SVM (Support vector Machine)

Traditional machine learning models, such as Random Forest and SVM, are widely used for log classification due to their robustness, interpretability, and efficiency in handling structured log data. These models are beneficial when deep learning approaches, like Transformers or LSTMs, may not be feasible due to limited data or computational constraints.

Random Forest is an ensemble learning technique that constructs multiple decision trees and aggregates their outputs to enhance classification accuracy while reducing overfitting. This makes it particularly effective for structured log classification tasks. The process begins with feature engineering, where structured attributes such as log level, event frequency, timestamps, and anomaly labels are extracted from the log data. Next, the dataset is split into training (80%) and testing (20%) subsets to ensure proper model validation. During model training, the Random Forest algorithm builds multiple decision trees, utilizing a specified number of estimators (n estimators) and employing either the entropy or Gini index as the splitting criterion. Once trained, the model is used for prediction and evaluation, where it classifies log events as either normal or anomalous. SVM is a powerful classification algorithm that is well-suited for structured log analysis, particularly in distinguishing between normal and anomalous logs. For Random Forest and SVM models in HDFS log analysis, the data collection process focuses on gathering structured logs, extracting relevant features, and preparing a labeled dataset for supervised learning-based classification. These models require numerical feature vectors rather than raw textual logs, necessitating appropriate preprocessing and transformation techniques. The feature extraction process begins, where log messages are converted into numerical representations using techniques such as TF-IDF (Term Frequency-Inverse Document Frequency) or

word embeddings to capture important textual patterns. Next, an appropriate kernel function (Linear, Radial Basis Function (RBF), or Polynomial) is selected to transform the log features into a higher-dimensional space, enabling better separation between normal and anomalous logs. During model training, the SVM classifier learns to maximize the margin between different log categories, ensuring robust separation as shown in Fig. 8.

Both RF and SVM are effective for traditional log classification, with RF performing slightly better due to its ability to handle complex feature interactions. However, SVM remains a strong choice for smaller datasets with well-defined feature spaces. These models are suitable for structured log analysis, where deep learning may not be necessary or feasible due to computational constraints.

Experimental result

This section provides the comparison between our Proposed Deep Learning Model (LSTM + Transformer) and Existing Models (Random Forest and SVM) on HDFS Log Dataset, CICIDS 2017, CSE-CIC-IDS2018 and UNSW-NB15 datasets concerning different evaluation metrics. Metrics include accuracy, precision, recall, F1-score, AUC-ROC, and inference time. Statistical analysis is also included to evaluate the performance enhancements of the suggested model statistically.

k-Fold Cross-Validation results

We split into five different training and testing using k-fold, specifying $k=5$ folds for cross-validation. Each fold is where the model performance is evaluated and the average performance metrics are calculated. This assists in evaluation of the robustness and reliability of the model on different splits of the data as represented in Table 5; Fig. 9.

Statistical analysis and p-value testing

We use a t-test to find the p-value for each metric to see if the differences in performance between the Proposed Model and Existing Models are statistically significant. If the p-value is less than 0.05, it means that the performance improvements are statistically significant represented in Table 6.

Attack detection performance

Subsequently, we assess the capability of the proposed model to identify various kinds of attacks, including distributed denial of service attacks, malware infections, and brute force attacks, and we compare the outcomes with those obtained from Random Forest and Support Vector Machines represented in Table 7.

Table 5 k-Fold Cross-Validation results (Average Accuracy, Precision, Recall, F1-Score)

Model	Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Proposed Model (LSTM + Transformer)	HDFS Log Dataset	98.2	96.5	97.8	97.1
	CICIDS 2017	96.8	95.0	94.5	94.8
	CSE-CIC-IDS 2018	97.1	97.2	96.8	97.0
	UNSW-NB15	98.0	96.8	97.4	97.1
Random Forest	HDFS Log Dataset	87.0	89.0	90.0	88.5
	CICIDS 2017	84.6	80.5	85.0	82.6
	CSE-CIC-IDS 2018	89.5	86.8	87.4	87.1
	UNSW-NB15	85.9	84.9	85.7	85.3
SVM	HDFS Log Dataset	88.0	87.5	88.4	87.9
	CICIDS 2017	85.5	84.3	83.0	83.6
	CSE-CIC-IDS 2018	87.3	84.0	85.5	84.7
	UNSW-NB15	86.4	83.6	84.9	84.2

Table 6 p-value (t-test) for statistical significance

Dataset	Metric	Proposed Model vs. Random Forest p-value	Proposed Model vs. SVM p-value
HDFS Log Dataset	Accuracy	0.001	0.002
	Precision	0.0015	0.002
	Recall	0.001	0.0015
	F1-Score	0.001	0.001
CICIDS 2017	Accuracy	0.002	0.003
	Precision	0.002	0.002
	Recall	0.003	0.004
	F1-Score	0.002	0.002
CSE-CIC-IDS 2018	Accuracy	0.004	0.003
	Precision	0.003	0.003
	Recall	0.005	0.004
	F1-Score	0.004	0.004
UNSW-NB15	Accuracy	0.0015	0.002
	Precision	0.002	0.002
	Recall	0.001	0.0015
	F1-Score	0.002	0.002

Multi-Class classification performance

Additionally, we evaluated the models based on multi-class classification, which is a classification method in which log entries are categorized into multiple categories, such as normal, suspicious, and malicious. The Proposed Model demonstrated consistently superior performance when it came to dealing with complex scenarios involving multiple classes.

Table 7 Attack detection rates for proposed model vs existing models

Dataset	Attack Type	Proposed Model (LSTM + Transformer)	Random Forest	SVM
HDFS Log Dataset	DDoS Attack	97.5%	80.2%	82.0%
	Malware Infection	98.0%	85.0%	83.5%
CICIDS 2017	Brute Force Attack	95.2%	81.5%	82.8%
	APT Attack	96.8%	88.3%	87.1%
CSE-CIC-IDS 2018	SQL Injection	98.3%	84.7%	85.5%

Table 8 Model robustness under different attack scenarios

Attack Scenario	Proposed Model (LSTM + Transformer)	Random Forest	SVM
Unseen Attack Detection	94.5%	75.0%	72.5%
Log Noise (10% Noise)	96.2%	88.1%	86.4%
Adversarial Attacks	92.7%	80.3%	79.2%

Model robustness

We introduced novel attack detection and evaluated the models under noisy conditions (with up to 10% noise in the logs) and adversarial settings in order to determine the robustness of the model as represented in Table 8.

Discussion

This section provides a comprehensive comparative evaluation of the proposed deep learning framework (LSTM + Transformer) against baseline models, namely Random Forest and SVM, across multiple benchmark datasets (HDFS Log, CICIDS 2017, CSE-CIC-IDS2018, and UNSW-NB15). Evaluation metrics include accuracy, precision, recall, F1-score, AUC-ROC, inference time, and robustness under adversarial conditions.

Cross-Validation and overall performance

The five-fold cross-validation results (Table 5) demonstrate that the proposed framework consistently outperforms Random Forest and SVM across all datasets. With average accuracies ranging from 96.8% to 98.2%, our model surpasses Random Forest (84.6%–89.5%) and SVM (85.5%–88.0%). Similar gains are observed for precision, recall, and F1-score. These improvements confirm the ability of hybrid sequential-contextual modeling (LSTM + Transformer) to capture richer log dependencies than traditional classifiers. Compared with related works such as DeepLog [Du et al.] (LSTM-based, ~95% detection accuracy) and LogBERT [Guo et al.] (Transformer-based, ~93–94% accuracy), our framework demonstrates superior performance by combining the strengths of both approaches. Furthermore, unlike prior

studies that focused exclusively on anomaly detection, our model integrates forensic correlation and visualization, thereby expanding its practical applicability.

Statistical significance

The t-test results (Table 6) indicate that the observed improvements are statistically significant ($p < 0.05$) across all datasets and metrics. For example, on UNSW-NB15, the proposed model achieves 98.0% accuracy compared to 85.9% (Random Forest) and 86.4% (SVM), with p-values below 0.002. This statistical validation underscores that the performance gains are not incidental but reflect a consistent improvement across data splits and contexts.

Attack detection performance

As shown in Table 7, the proposed model demonstrates stronger attack-type detection capabilities. For instance, it detects SQL Injection attacks with 98.3% accuracy, significantly outperforming Random Forest (84.7%) and SVM (85.5%). Similarly, for Advanced Persistent Threat (APT) attacks in CSE-CIC-IDS2018, the model achieves 96.8% detection, compared to $< 89\%$ for baselines. These results are aligned with prior findings in intrusion detection [14], but with higher robustness across diverse datasets and attack types.

Multi-Class classification

Multi-class classification analysis further highlights the adaptability of the proposed framework. Unlike Random Forest and SVM, which show a drop in performance when moving from binary to multi-class settings, our model maintains high accuracy ($> 95\%$) and F1-score ($> 94\%$). This advantage is particularly critical in real-world SOC, where anomalies are not limited to binary malicious/benign categories.

Robustness analysis

Robustness testing (Table 8) reveals that the proposed model is more resilient to unseen attacks, noisy logs, and adversarial scenarios. For example, under adversarial conditions, it retains 92.7% detection accuracy, compared to 80.3% (Random Forest) and 79.2% (SVM). This robustness aligns with recent works advocating hybrid deep models for adversarial resilience [24], but our system demonstrates higher generalization across datasets.

Implications and comparison with literature

In summary, the proposed model not only outperforms classical machine learning baselines such as Random Forest and SVM across all evaluation metrics but also advances beyond prior deep learning approaches like DeepLog and LogBERT by achieving higher accuracy and integrating forensic correlation. The performance improvements are statistically validated with p-values

less than 0.05, underscoring their significance. Moreover, the model demonstrates robustness against noise and adversarial scenarios, an aspect seldom explored in earlier studies, and further enhances its practical value by offering real-time dashboards that bridge academic research with Security Operations Center (SOC) practice. These results highlight the novel contribution of combining LSTM and Transformer architectures in a unified pipeline, yielding measurable improvements in accuracy, reliability, and forensic applicability.

The system automates forensic correlation across multiple log sources, enabling reconstruction of attack timelines. This automation reduces manual effort and improves the accuracy of forensic investigations. It addresses limitations of manual methods, which are often slow and error-prone, the real-time visualization dashboard supports proactive incident response by providing immediate insights into anomalies, enhancing situational awareness. Despite these strengths, challenges remain. The volume and heterogeneity of logs demand scalable solutions. Model adaptability to emerging threats requires ongoing retraining. Future research should explore advanced multi-source log fusion techniques, such as Graph Neural Networks, to improve comprehensive analysis and forensic accuracy. Incorporating recent developments in deep learning—like hybrid models combining CNNs with Transformers—can further enhance detection and interpretability [4, 24]. This integrated framework improves detection precision and forensic efficiency. It transforms traditional post-incident analysis into a proactive, automated process, strengthening cybersecurity resilience. Continued innovation in scalable, adaptive deep learning models will be vital for handling future complexities in log analysis.

Conclusion and future directions

This study demonstrates the effectiveness of deep learning models, particularly LSTM and Transformer-based architectures such as BERT and LogBERT, in advancing log-based anomaly detection and forensic analysis. Experimental results showed that the proposed framework achieved high accuracy, with LSTM models reaching up to 98.2% and BERT-based models up to 94.2%, significantly outperforming traditional approaches like Random Forest and SVM. These results confirm the ability of deep learning to capture both temporal dependencies and contextual relationships within system logs, enabling more precise anomaly detection, reduced false positives, and improved forensic reconstruction.

Looking ahead, future research should focus on addressing scalability, adaptability, and interpretability challenges. Integrating Graph Neural Networks (GNNs) could enhance multi-source log correlation and forensic accuracy, while hybrid frameworks combining deep

learning with heuristic methods may further improve resilience and reduce false positives. Additionally, optimizing models for real-time inference and incorporating explainable AI techniques will be essential to support trust, transparency, and practical deployment in enterprise-scale cybersecurity environments.

Acknowledgements

The author extend their appreciation to Taif University, Saudi Arabia, for supporting this work through project number (TU-DSPP-2024-17).

Authors' contributions

Author Contributions: Leeladhar Chourasiya, Sushma Khatri, Umesh Kumar Lilhore, Sarita Simaiya, and MD Monish Khan contributed significantly to the conceptualization, methodology, data collection, analysis, and manuscript preparation. Roobaee Alroobaee, Abdullah M. Baqasah, and Majed Alsafyani played a key role in revising the manuscript, providing funding, and assisting with the rewriting of the draft. Their combined efforts ensured the successful completion of the study and its publication.

Funding statement

This research was funded by Taif University, Taif, Saudi Arabia project number (TU-DSPP-2024-17).

Data availability

The dataset will be available with the corresponding author based on individual requests.

Declarations

Ethics approval and consent to participate

Not applicable.

Consent for the publication

Not applicable.

Competing interests

The authors declare no competing interests.

Received: 5 June 2025 / Accepted: 29 September 2025

Published online: 28 October 2025

References

1. Fu, Ying, Meng Yan, Jian Xu, Jianguo Li, Zhongxin Liu, Xiaohong Zhang, and Dan Yang (2022) "Investigating and improving log parsing in practice." In proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering, pp 1566–1577 <https://doi.org/10.1145/3540250.3558947>
2. He, Pinjia, Jieming Zhu, Zibin Zheng, Michael R. Lyu (2017) "Drain: An online log parsing approach with fixed depth tree." In 2017 IEEE international conference on web services (ICWS), pp 33–40 IEEE. <https://doi.org/10.1109/ICWS.2017.13>
3. Chen, Rui, Shenglin Zhang, Dongwen Li, Yuzhe Zhang, Fangrui Guo, Weibin Meng, Dan Pei, Yuzhi Zhang, Xu Chen, Yuqing Liu (2020) "Logtransfer: cross-system log anomaly detection for software systems with transfer learning." In 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE), pp 37–47 IEEE. <https://doi.org/10.1109/ISSRE5003.2020.00013>
4. Li, Qiong, Xiaotong Liu, Xuecai Hu, Md Atiqur Rahman Ahad, Min Ren, Li Yao, Yongzhen Huang (2025) "Machine learning-based prediction of depressive disorders via various data modalities: a survey." IEEE/CAA Journal of Automatica Sinica 12, 7:1320–1349. <https://doi.org/10.1109/JAS.2025.125393>
5. Yan Bo, Cheng Yang, Chuan Shi, Yong Fang, Qi Li, Yanfang Ye, and Junping Du (2023) "Graph mining for cybersecurity: A survey." ACM transactions on knowledge discovery from data 18, 2:1–52. <https://doi.org/10.1145/36102286>
6. Shen, Junjie, Ranran Tie, Zujin Li, Bocheng Liu, Zhihui Fan, and Jingya Lu (2024) "Neural network-based log anomaly detection algorithm for 6G wireless integrated cyber-physical system." Wireless personal communications, pp 1–19 <https://link.springer.com/article/10.1007/s11277-024-11218-9>
7. Lee, Yookyung, Jina Kim, Pilsung Kang (2023) "Lanobert: System log anomaly detection based on bert masked language model." Applied Soft Computing 146:110689. <https://doi.org/10.1016/j.asoc.2023.110689>
8. Chen, Long, Yanting Wang, Qiaojuan Wang, Yanqing Song, and Jianguo Chen (2025) "Cybersecurity multi-dimensional few-shot data generation on malicious enhancement." IEEE Transactions on dependable and secure computing. <https://doi.org/10.1109/TDSC.2025.3558545>
9. Karthick R (2025) Context-aware topic modeling and intelligent text extraction using transformer-based architectures. <https://doi.org/10.2139/ssrn.5275391>
10. Natarajan, Arul Kumar, Mohammad Gouse Galety, Shirin Noekhah, Raja-sundaran Soundararajan, Sobirov Xurshed, and Zikiryoyev Xasan (2024) "Enhancing cybersecurity in smart grid systems through advanced log file analysis with machine and deep learning techniques." In 2024 Third International Conference on Sustainable Mobility Applications, Renewables and Technology (SMART) pp 1–10. IEEE. <https://doi.org/10.1109/SMART63170.2024.10815479>
11. Soltani, Mahdi, Behzad Ousat, Mahdi Jafari Siavoshani, Amir Hossein Jahangir (2023) "An adaptable deep learning-based intrusion detection system to zero-day attacks." J Inf Secur Appl. 76:103516. <https://doi.org/10.1016/j.jisa.2023.103516>
12. Hussein, Safwan Mawlood, and Abubakar Muhammad Ashir (2024) "Machine Learning-Driven Intrusion Detection Systems: Reducing False Alarms and Enhancing Accuracy." EURASIAN JOURNAL OF SCIENCE AND ENGINEERING 10, 3:85–96. <https://doi.org/10.23918/eajse.v10i3p9>
13. Han, Shangbin, Qianhong Wu, Han Zhang, Bo Qin, Jiankun Hu, Xingang Shi, Linfeng Liu, and Xia Yin (2021) "Log-based anomaly detection with robust feature extraction and online learning." IEEE Transactions on Information Forensics and Security 16, 2300–2311. <https://doi.org/10.1109/TIFS.2021.3053371>
14. Svacina, Jan, Jackson Raffety, Connor Woodahl, Brooklyn Stone, Tomas Cerny, Miroslav Bures, Dongwan Shin, Karel Frajtek, Pavel Tisnovsky (2020) "On vulnerability and security log analysis: a systematic literature review on recent trends." In Proceedings of the International Conference on Research in Adaptive and Convergent Systems, pp 175–180. <https://doi.org/10.1145/3400286.3418261>
15. Chu G, Wang J, Qi Q, Sun H, Tao S, Liao J (2021) Prefix-Graph: A versatile log parsing approach merging prefix tree with probabilistic graph. 2021 IEEE 37th International Conference on Data Engineering (ICDE), Chania, Greece, 2411–2422. <https://doi.org/10.1109/ICDE51399.2021.00274>
16. Ma, Junchen, Yang Liu, Hongjie Wan, and Guozhi Sun (2023) "Automatic parsing and utilization of system log features in log analysis: a survey." Applied Sciences 13, 8:4930. <https://doi.org/10.3390/app13084930>
17. Huo Y, Su Y, Lee C, Lyu MR (2023) SemParser: A semantic parser for log analytics. 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), Melbourne, Australia, 881–893 <https://doi.org/10.1109/ICSE48619.2023.00082>
18. J. Zhou, Y. Qian, Q. Zou, P. Liu and J. Xiang (2022) "DeepSyslog: deep anomaly detection on syslog using sentence embedding and metadata," in IEEE transactions on information forensics and security, vol. 17, pp 3051–3061. <https://doi.org/10.1109/TIFS.2022.3201379>
19. He, Shilin, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu (2021) "A survey on automated log analysis for reliability engineering." ACM computing surveys (CSUR) 54, no.6:1–37. <https://doi.org/10.1145/3460345>
20. Uetz R, Hemminghaus C, Hackländer L, Schlipper P, & Henze M (2021) Reproducible and adaptable log data generation for sound cybersecurity experiments. In Proceedings of the 37th annual computer security applications conference pp 690–705. <https://doi.org/10.1145/3485832.3488020>
21. Fu, Ying, Meng Yan, Jian Xu, Jianguo Li, Zhongxin Liu, Xiaohong Zhang, and Dan Yang (2022) "Investigating and improving log parsing in practice." In Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering, pp 1566–1577. <https://doi.org/10.1145/3540250.3558947>
22. Apruzzese, Giovanni, Pavel Laskov, Edgardo Montes de Oca, Wissam Mal-louli, Luis Brdalo Rapa, Athanasios Vasileios Grammatopoulos, and Fabio Di Franco (2023) "The role of machine learning in cybersecurity." Digital threats: research and practice 4, no.1:1–38. <https://doi.org/10.1145/3545574>
23. Liu, Jung-Chun, Chao-Tung Yang, Yu-Wei Chan, Endah Kristiani, and Wei-Je Jiang (2021) "Cyberattack detection model using deep learning in a network log system with data visualization." The Journal of Supercomputing 77,

no.10:10984–11003. <https://link.springer.com/article/10.1007/s11227-021-03715-6>

24. Yu, Boxi, Jiayi Yao, Qiulai Fu, Zhiqing Zhong, Haotian Xie, Yaoliang Wu, Yuchi Ma, and Pinjia He (2024) "Deep learning or classical machine learning? an empirical study on log-based anomaly detection." In Proceedings of the 46th IEEE/ACM international conference on software engineering, pp 1–13. <https://doi.org/10.1145/3597503.3623308>
25. Choudhary C, Vyas N, Umesh Kumar L (2023) An optimized sign language recognition using convolutional neural networks (CNNs) and tensor-flow. In

(2023) *3rd International Conference on Technological Advancements in Computational Sciences (ICTACS)*, pp 896–901. IEEE, <https://doi.org/10.1145/3597503.3623308>

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.